

Trabajo Práctico Integrador

Alumnos - Grupo_141

Maknis Luciano

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación I

28 Junio de 2026

Tabla de contenido

Marco teórico	3
Decisiones Técnicas y Arquitectura	4
Dificultades y conclusiones	6
Referencias	7

Marco teórico

Estructura general

La estructura visual del menú principal consiste en un bucle "while" el cual por su naturaleza es óptimo para un menú el cual se desconoce cuántas veces será transitado por el usuario, en él se encuentran las distintas opciones la cuales llevan a funciones "if" anidadas para direccionar al usuario a la opción requerida. En las susodichas funciones anidadas se reclutan funciones propias, resultando en su conjunto en una estructura de bucle en la cual no se desarrolla el grueso de la lógica, sino que funciones previamente desarrolladas son convocadas. En forma previa a la estructura del bucle se encuentran las funciones las cuales cumplen tareas tanto comunes como específicas de cada opción. Muchas de estas funciones utilizan variables globales, como la lista que replica al archivo CSV, lo cual facilita enormemente la creación de funciones.

Estructura de datos

Se seleccionó a la lista como la estructura de datos predominante en la resolución del trabajo, debido a la similitud de su anatomía con la de un archivo CSV por lo cual ofrecía una facilidad para la visualización, manipulación y sobre escritura de datos. Cada renglón del archivo CSV puede representar una lista, por eso se recurrió a una lista de listas.

El diccionario, con una anatomía diferente, fue utilizado en situaciones específicas en donde debían declararse y utilizarse múltiples variables, permite asociar de forma directa una clave única, como el nombre del continente, con un valor que puede variar, como cantidad de países, evitando uso de variables sueltas. Otorga un mejor orden y legibilidad estética a pesar de su mayor complejidad para operarlo.

Modularización

El beneficio del uso de funciones propias se pudo observar en tareas repetitivas y comunes a distintas funcionalidades principalmente validar entradas del usuario y normalizar entradas del usuario con elementos del archivo CSV. También otras funciones cumplen tareas específicas de cada opción la cuales se usan una única vez, anulando su efecto de ahorro en tareas repetitivas.

Algoritmos de Ordenamiento

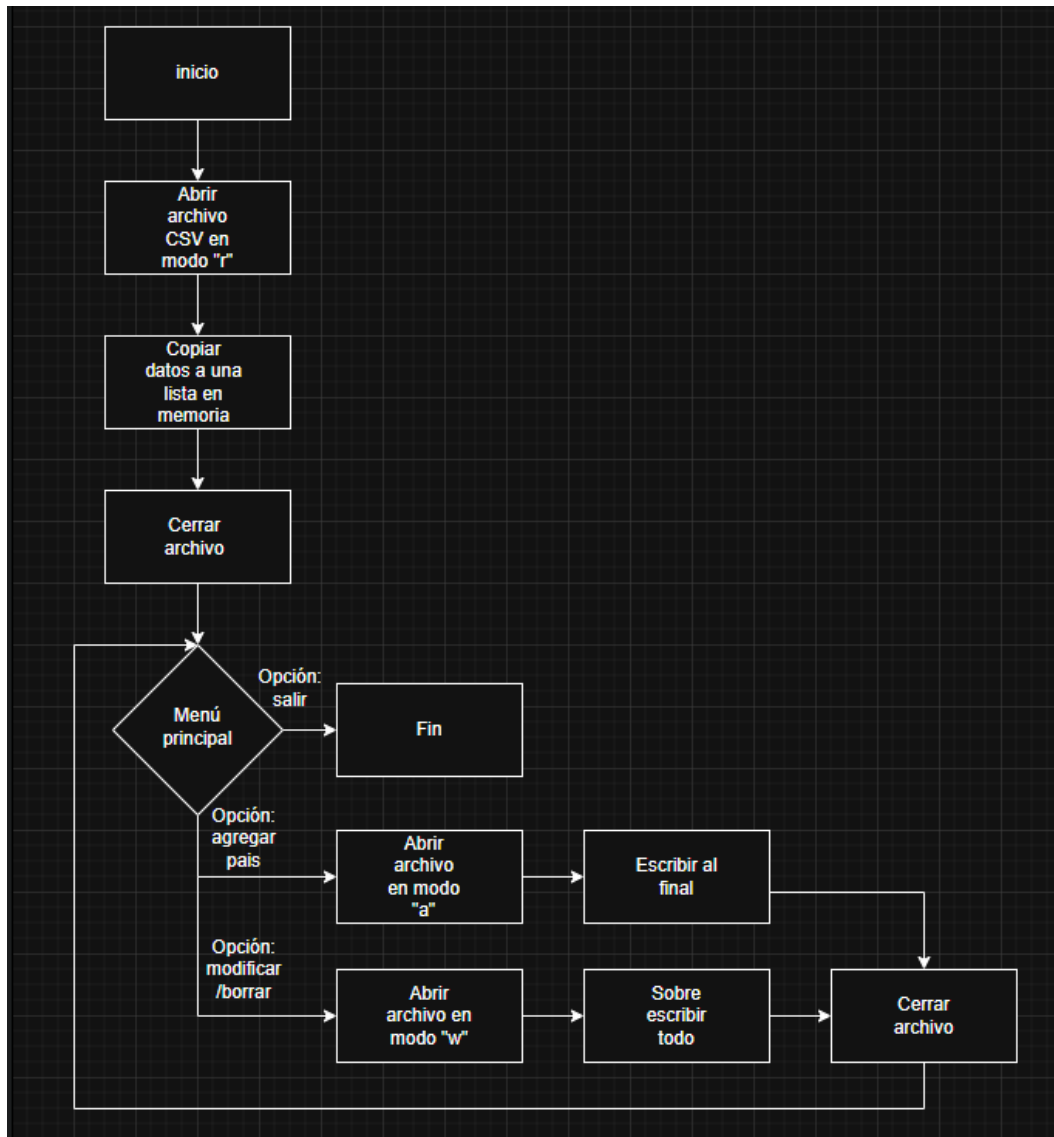
En situaciones como ordenar alfabéticamente elementos de la lista se optó por la función nativa "sorted()", esta ofrecía la simpleza de no demandar gran cantidad de recursos mentales para lograr el objetivo. Mientras en situaciones como el ordenar de mayor a menor o viceversa, con mayor complejidad y de forma manual se debió implementar el algoritmo burbuja para lograr cumplir el objetivo, en este además se procedió a aislar el encabezado mediante slicing de forma "[1:]" para procesar únicamente los registros numéricos necesarios.

Decisiones técnicas y arquitectura

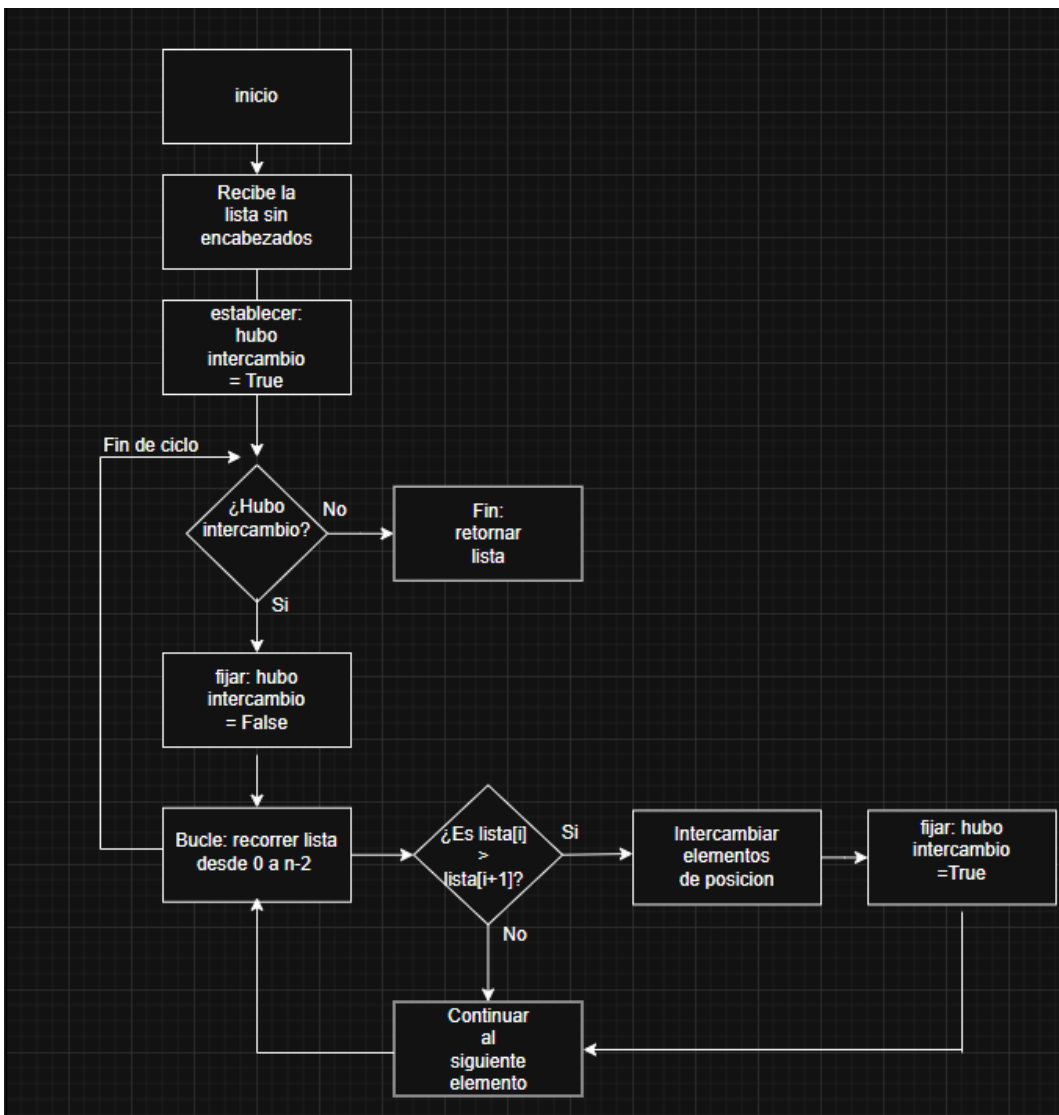
Decisiones Técnicas y Arquitectura

Entre los desafíos técnicos a superar por la estructura se encuentra el normalizar las entradas validas del usuario con valores del archivo CSV, los cuales pueden presentar discrepancias como espacios, tildes o mayúsculas. Con una función propia es posible igualar textos y lograr el correcto funcionamiento del programa.

Para utilizar los datos del archivo CSV en forma segura, se utiliza una estrategia de solo manipularlos de ser estrictamente necesario. Mediante una función "with open" en modo "r" se copian los datos a una lista y esta es la que se opera durante el proceso. Posteriormente las entradas del usuario validas que conforman nuevos elementos de la lista son agregadas al final del archivo con una una función "with open" en modo "a". Los nuevos datos que alteran valores de elementos que ya existentes de la lista se suman al archivo sobre escribiéndolo por completo con una función "with open" en modo "w".



El algoritmo burbuja utilizado en el programa recibe la lista sin encabezados y establece la bandera para detener el proceso si bien la lista se encuentra ordenada. Cuando no hay intercambios significa que la lista se encuentra ordenada. Recorre los elementos de la lista, en caso de buscar ordenar los mayores al final, intercambia el elemento actual con el siguiente si este es mayor, luego de sucesivos recorridos cuando ya no hay intercambios devuelve la lista ordenada.



Dificultades y conclusiones

Dificultades

La resolución del trabajo estuvo repleta de desafíos y dificultades, entre las más destacadas se encuentra la implementación manual del algoritmo burbuja, el cual trajo aparejados inconvenientes como el programa teniendo que comparar números, pero estos contar con el tipo de dato por defecto "string", adicionalmente, luego de muchos intentos no se lo pudo hacer funcionar sin crear una nueva lista sin encabezados. Con una demanda intelectual mayor podría haberse buscado la forma de unificar la función que realizan el algoritmo. La transición de la forma de programación principiante que solo resuelve la consigna a utilizar funciones propias y buscar la manera de estas interactuar para resolver los objetivos requeridos de forma modular, resulto en el mayor desafío del proyecto.

Conclusiones

El manejo de archivos, datos complejos y modularización mediante funciones propias representaron enormes desafíos pensando y experimentando su lógica a la hora de su implementación. Lograr desarrollar el trabajo transitando estos desafíos represento un cambio visible. El trabajo y todos los anteriores con sus dificultades y desafíos presentaron un gran aprendizaje como primeros pasos, comenzando desde cero, en la programación.

Referencias

Bibliografía / Webgrafía: Material de lectura aula virtual Programación I

Diagramas de flujo realizados en el sitio <https://app.diagrams.net/>

Repositorio del proyecto <https://github.com/Lucho10999/Trabajo-Practico-Integrador--Programacion-I/tree/main>

Video explicativo

<https://drive.google.com/file/d/17JEvIKe0OFDeIWafot31pxZjYZKDqh9U/view?usp=sharing>